HW8

- **Submission date: 04/22/2025 (11:59 PM)**
- Read your class notes *thoroughly* before attempting the assignment.
- You can submit ***well commented*** code in either `.py` or `.ipynb` formats.
- Submitting a PDF explaining your code is *highly* encouraged.

1. **Feedforward Networks** (please read the TensorFlow tutorials uploaded by Joanna and Chapter 10 in the textbook). We will explore training multi-layer feedforward networks with two examples.

    a. Sentiment analysis on the **IMDb** dataset.
    b. Classifying images in the **CIFAR-10** dataset.

2. **Gradient Descent**. Consider the function $f(x_1, x_2) = x_1^2 + 25x_2^2$. You are required to estimate the parameters of this function (i.e., $[1, 25]$) by completing the following tasks.

    a. Visualize the function: Using NumPy and MatplotLib, draw the contour plot of $f(x_1, x_2)$. This function is parameterized by $\boldsymbol{w}^* = [w_1^*, w_2^*] = [1, 25]$.
        i. You can use the following ranges for both coordinates for the plot: $x_i \in [-10, 10]$ (500 points should suffice).
        ii. The MatplotLib function `contour()`, and the NumPy function `meshgrid()` should come in handy.
    b. Noisy observations: generate $1{,}000$ noisy observations of the function, where the observations of the function are defined by $y = f(x_1, x_2) + \mathcal{N}(0, \sigma^2)$.
        i. Here, the noise is Gaussian with variance = 1.5. Set NumPy's `random_seed = 101` to ensure reproducibility.
        ii. Generate the data set using 1,000 independent random samples of $x_1 \sim Uniform([0,3])$ and $x_2 \sim Uniform([0,1])$.
    c. Gradient descent trajectories: generate the trajectories of (a) batch gradient descent, (b) minibatch gradient descent (use minibatch size = 20), and (c) stochastic gradient descent. Plot the trajectories superimposed on the contours of loss function of batch gradient descent.
        i. For simplicity, assume you have a model of the form $\hat{y} = w_1 x_1^2 + w_2 x_2^2$, and are using squared loss $(y - \hat{y})^2$ as the loss function.
        ii. Remember that minibatches are chosen *randomly* in each iteration with replacement.
        iii. Use $[1.0, 1.0]$ as the initial condition and $\eta = 4 \times 10^{-2}$ for batch and minibatch GD. For SGD, use $\eta = 5 \times 10^{-4}$ (to prevent excessive dithering)
    d. Convergence: how many iterations does each of the three algorithms require to come within an accuracy of 10^{-3}? That is, after how many iterations is $\|\boldsymbol{w}_t - \boldsymbol{w}^*\|_2 < 10^{-3}$ for each of the three algorithms, where $\boldsymbol{w}^* = [1, 25]$?
    e. **(Ungraded)** Play around with the initial conditions and η. Can you find a value of η for which the Batch GD trajectory is on the brink of instability (i.e., shows oscillations)?

3. **Gradients and Chain Rules.** Find the gradient of the following functions at the given points (you might have to use some of the chain rules discussed in class).

    a. $f(x, y, z) = (2x^2 + \cos(yz) - 3z)^2$ at $[x, y, z] = [2, \pi, 1]$
    b. $f(x, y, z) = \cos(x)\cosh(y)$, where $\cosh(y) = \frac{e^y + e^{-y}}{2}$ at $[x, y] = [\frac{\pi}{2}, \ln(2)]$.

    Note that both functions above are functions of three variables x, y, z.